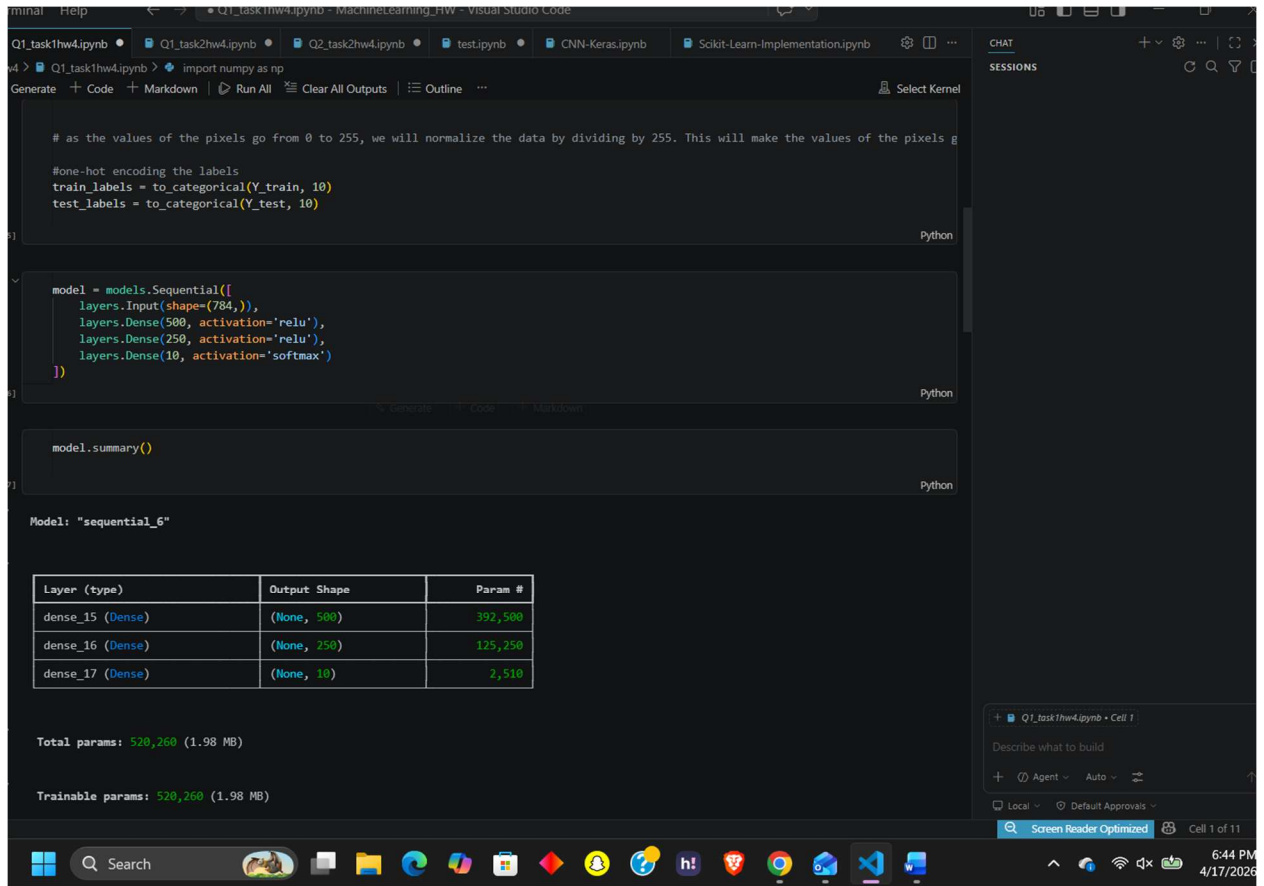


HW 4 Report

1) Task 1:



The screenshot shows a Jupyter Notebook with the following code and output:

```
# as the values of the pixels go from 0 to 255, we will normalize the data by dividing by 255. This will make the values of the pixels g

#one-hot encoding the labels
train_labels = to_categorical(Y_train, 10)
test_labels = to_categorical(Y_test, 10)

model = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(500, activation='relu'),
    layers.Dense(250, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.summary()
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
dense_15 (Dense)	(None, 500)	392,500
dense_16 (Dense)	(None, 250)	125,250
dense_17 (Dense)	(None, 10)	2,510

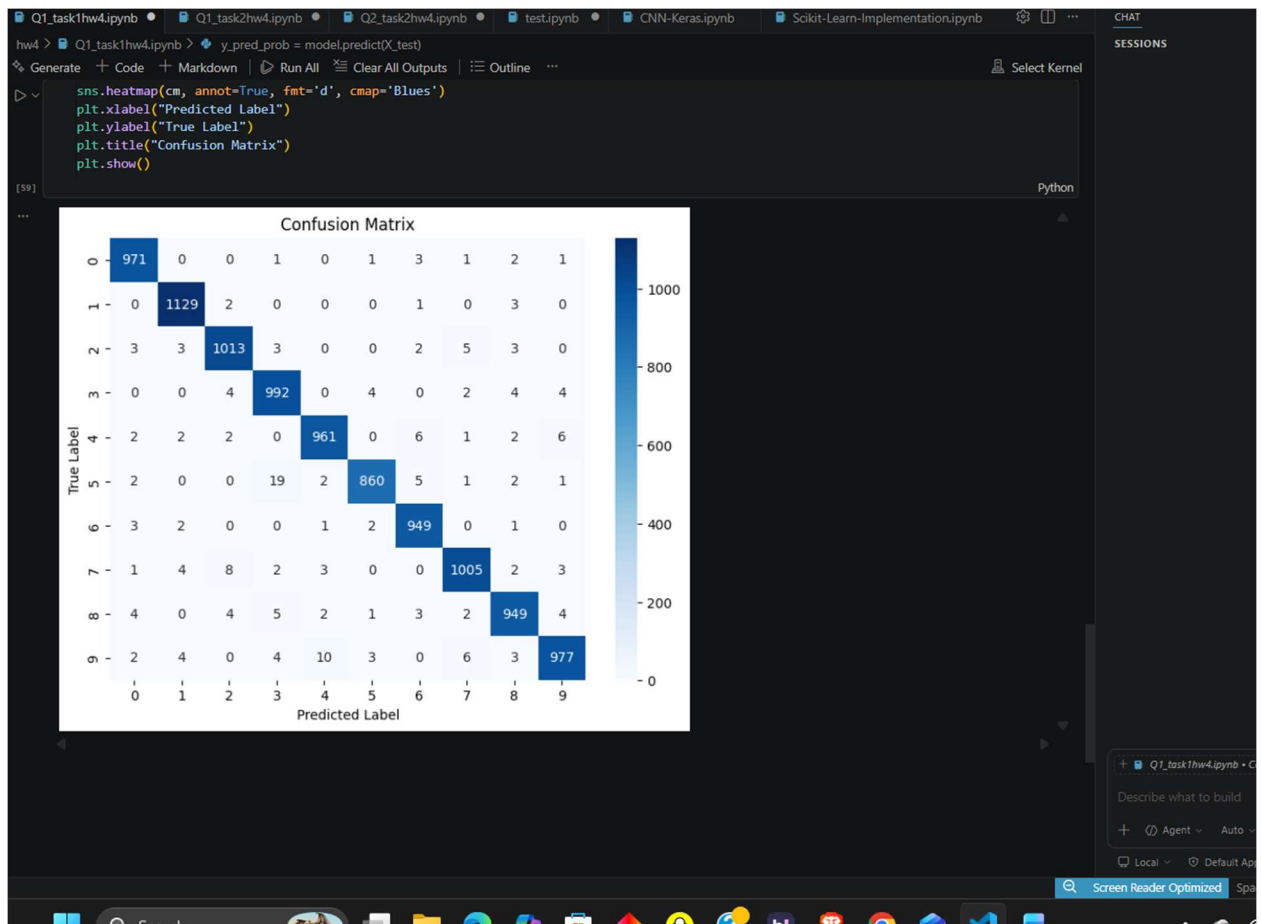
Total params: 520,260 (1.98 MB)

Trainable params: 520,260 (1.98 MB)

The Model That I made had two hidden layers, one output layer with a softmax function.

It gave me a test accuracy of 0.9805999994277954 which was a good accuracy.

My confusion matrix that I got was:



Which showed that I was able to classify 1,... 10 correctly most of the time, but had some trouble (comparatively) classifying 3 and 5 (19 error), (They look similar) also 4 and 9 (They also look similar) (10 errors).

Task 2:

To implement PCA, I imported PCA from sklearn module,

The screenshot shows a Jupyter Notebook with the following code and outputs:

```
hw4 > Q1_task2hw4.ipynb > # Observations:
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | Python (ML_HW) (Python 3.10.11)
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# Flatten images: 28x28 -> 784
X_train = X_train.reshape(60000, 784)
X_test = X_test.reshape(10000, 784)

# Apply PCA: reduce 784 -> 50
pca = PCA(n_components=50)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

[16] ✓ 0.5s Python

```
print("Original X_train shape:", X_train.shape)
print("PCA X_train shape:", X_train_pca.shape)
print("Original X_test shape:", X_test.shape)
print("PCA X_test shape:", X_test_pca.shape)
```

[17] ✓ 0.0s Python

... Original X_train shape: (60000, 784)
PCA X_train shape: (60000, 50)
Original X_test shape: (10000, 784)
PCA X_test shape: (10000, 50)

```
train_labels = to_categorical(Y_train, 10)
test_labels = to_categorical(Y_test, 10)
```

[18] ✓ 0.0s Python

```
model_pca = models.Sequential([
    layers.Input(shape=(50,)),
    layers.Dense(500, activation='relu'),
    layers.Dense(250, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

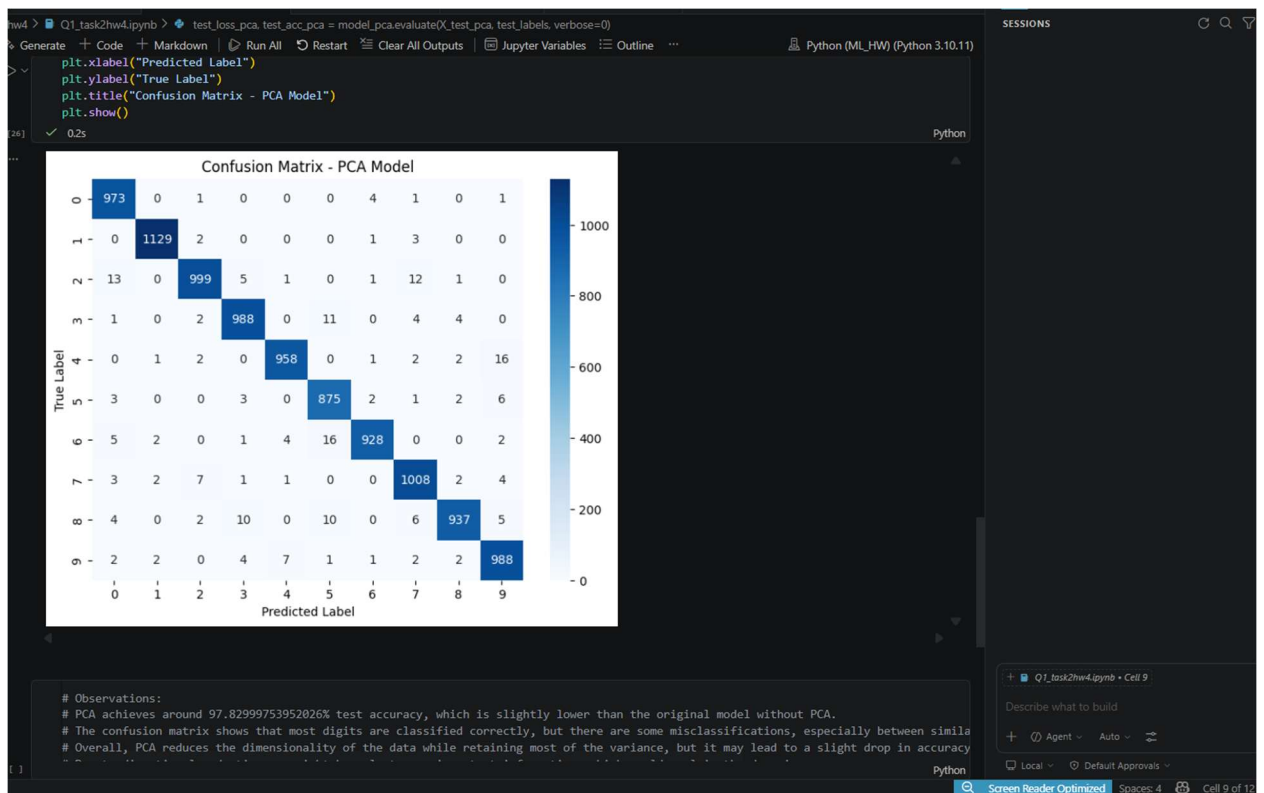
[19] ✓ 0.0s Python

On the right, there is a CHAT panel with the text "Describe what to build" and a "Screen Reader Optimized" button. The bottom status bar shows the time as 6:47 PM on 4/17/2026.

With PCA,

PCA Model Test Accuracy: 0.9782999753952026 which is good but a little lesser than without PCA,

PCA confusion Matrix:



Task 3:

Observations:

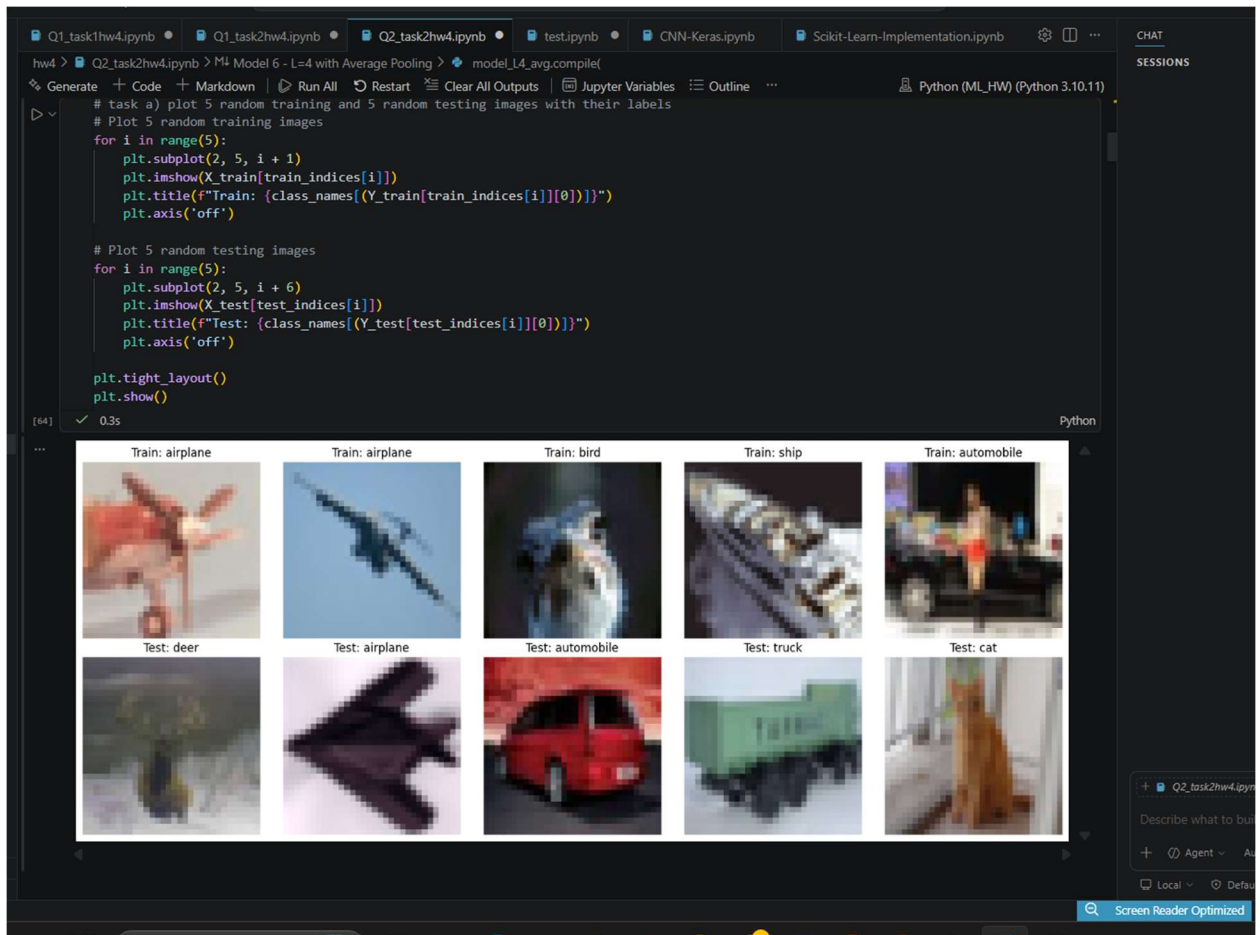
PCA achieves around 97.82999753952026% test accuracy, which is slightly lower than the original model without PCA.

The confusion matrix shows that most digits are classified correctly, but there are some misclassifications, especially between similar digits (e.g., 4 and 9).

Overall, PCA reduces the dimensionality of the data while retaining most of the variance, but it may lead to a slight drop in accuracy compared to using the full feature set. Due to dimensional reduction, we might have lost some important information, which could explain the drop in accuracy.

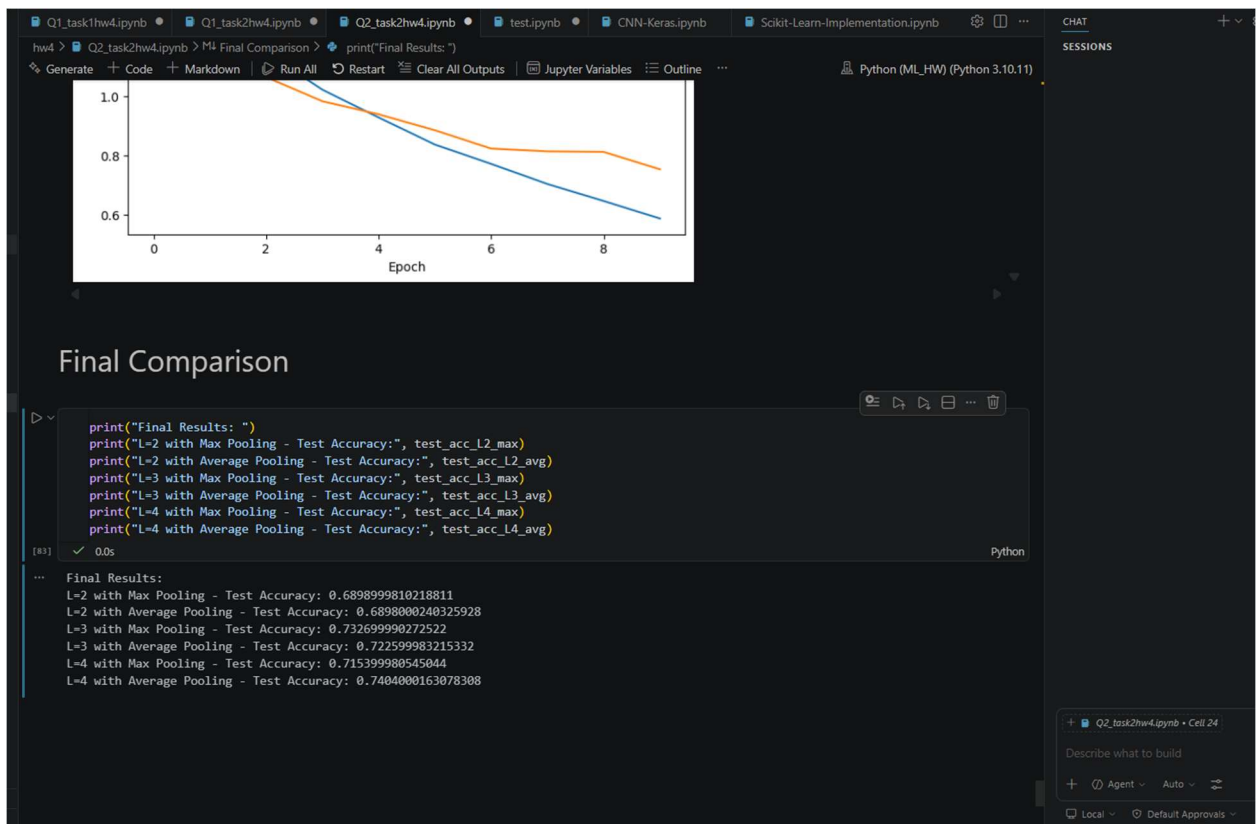
2) A)

5 random images inside the training set and 5 random images inside the testing set:



b)

We got graphs of all the different model layers (2-4) with both avg and max pooling.
And for final analysis:



c)

Based on testing accuracy, The best setting was CNN with L=4, with average pooling, as it had the highest accuracy among all other settings.

With results being Test Accuracy~ 0.74 which makes sense as increased layers made it possible to better classify the images.